

Lecture 11 - Statistical Learning

Alice Wu

UW Madison Econ 695

- *Statistical Learning* is a set of techniques for modeling relationships between *inputs* (or predictors) and *outputs* (or responses)
 - want to form *good predictors* of the output, rather than evaluate causality
 - want *data driven* procedures for model development that can be automated
- Example tasks:
 - “classification” (assign best guess of gender)
 - “prediction” (assign best guess of value of a click)
 - “clustering” (group unstructured data based on features)

- SL (or “machine learning”, ML) is a key input to AI processes. E.g., a large online retailer has to decide what items in a catalog to drop, but too many items to monitor by hand $\Rightarrow$ need models that:
 - predict $\Delta\pi$ (profit) on item if dropped
 - predict $\Delta\pi$ for close substitute products
 - predict Δ future customer revenues, associated $\Delta\pi$
 - combine 3 predictions and drop if sum of profit changes is negative.

Supervised versus Unsupervised

SL can be classified into two categories: supervised vs. unsupervised learning.

- Supervised Learning: predict Y using features X
 - Regression
 - Classification: $y_{gi} = 1$ if $i \in \text{group } g$
- Unsupervised Learning: only have access to inputs/features X , no target output Y .
 - Data visualization/pre-processing: reducing the dimension of X via principal component analysis.
 - Discover unknown subgroups in data: clustering.

We will start with supervised learning, and introduce the following:

- Bias-variance tradeoff
- Out-of-sample fit, Cross Validation
- Regularization (penalty for extra parameters); Shrinkage - Ridge, Lasso

Framework: 1-dimensional response

- y = response;
- x = “features” (implicit “ i ” subscript)
- $f(x) = E[y|x]$ is the benchmark: can write

$$y = f(x) + \epsilon \text{ where } E[\epsilon|x] = 0$$

- Recall: $f(x) = E[y|x]$ minimizes $E[(y - g(x))^2]$. And $E[\epsilon \times h(x)] = 0$ for any function h of x .
- In practice we have to work with predictor $\hat{f}(x)$ of the CEF. $\hat{f}$ incorporates 2 things: choice of function; estimation error. E.g. $\hat{f}$ = polynomial of order p :

$$y = b_0 + b_1x + b_2x^2 + \dots + b_px^p + v$$

$$\hat{f}^p(x) = \hat{b}_0 + \hat{b}_1x + \hat{b}_2x^2 + \dots + \hat{b}_px^p$$

- 2 sources of error:
 - ① deviation of $\hat{f}^p(x)$ from $f(x)$ with ∞ sample: bigger p is better for $\downarrow$ bias
 - ② deviation of $\hat{f}^p(x)$ from $E[\hat{f}^p(x)]$: smaller p is better for $\downarrow$ variance
- We want a predictor $\hat{f}$ that achieves low variance & low bias. Fitting a higher-order polynomial may reduce bias but increase variance.

Bias-Variance Tradeoff

- Given a fitted model $\hat{f}$ we can forecast y_0 associated with x_0

$$y_0 - \hat{f}(x_0) = \underbrace{y_0 - f(x_0)}_{\epsilon} + \underbrace{f(x_0) - E[\hat{f}(x_0)]}_{\text{bias}} + \underbrace{E[\hat{f}(x_0)] - \hat{f}(x_0)}_{\text{noise}}$$

- Suppose we can randomly draw a training data set repeatedly from the population.
- In each training set, we will fit a model $\hat{f}$. And the expectation $E[\hat{f}(x_0)]$ represents the mean of estimates of $f(x_0)$ across all the training sets.
- Decompose Mean Squared Error: $E[(y_0 - \hat{f}(x_0))^2]$ at x_0 :

$$\begin{aligned} E[(y_0 - \hat{f}(x_0))^2] &= E[\epsilon^2] + \left(f(x_0) - E[\hat{f}(x_0)] \right)^2 + \\ &\quad + E[(E[\hat{f}(x_0)] - \hat{f}(x_0))^2] \\ &= \underbrace{\text{var}(\epsilon)}_{\text{irreducible}} + \underbrace{\text{Bias}(\hat{f}(x_0))^2}_{\text{squared bias}} + \underbrace{\text{var}(\hat{f}(x_0))}_{\text{variance}\hat{f}} \end{aligned}$$

- Variance of $\hat{f}$:** the amount by which $\hat{f}$ would change if we estimate the model using a different training data set.
- Bias of $E[\hat{f}]$:** the error introduced by approximating the CEF/ “a real-life problem” by a much simpler model. E.g., using a linear regression to approx. a complicated underlying model.

Example:

- Polynomial example: approximate $f(x)$ by $\hat{f}(x) = \hat{b}_0 + \hat{b}_1x + \hat{b}_2x^2 + \dots + \hat{b}_px^p$. More flexible model with a higher p will have lower bias, but may be “noisier”.
- Simulation (see Jupyter notebook):
 - $x = 1, 3, 5, 7 \dots 29$ (15 observations, made 100 data sets)
 - (DGP) $y = x + \epsilon$, $\epsilon \sim N(0, 3)$. The CEF function is $f(x) = x$.
 - fit linear and quadratic models:

$$\text{(linear) } y = \alpha_0 + \alpha_1x + u$$

$$\text{(quadratic) } y = \beta_0 + \beta_1x + \beta_2x^2 + v$$

- At $x = x_0$:

$$\text{bias} = f(x_0) - E[\hat{f}(x_0)]$$

$$\text{variance} = E[(\hat{f}(x_0) - E[\hat{f}(x_0)])^2]$$

Example:

- Jupyter notebook: we draw 100 samples, and show:
 - “bias1” vs “variance1” from linear $\hat{f}$
 - “bias2” vs “variance2” for quadratic $\hat{f}$.
- Case 1: linear CEF – quadratic $\hat{f}$ show similar bias but larger variance!

(a) Linear CEF

$$f(x) = x$$

```
x0= 15  
sum1 = summary(training1)  
sum1.loc[sum1['x']==x0,['mse1','var1','bias1','mse2','var2','bias2']] # if want latex: .to_latex()  
✓ 0.0s
```

	mse1	var1	bias1	mse2	var2	bias2
7	7.171607	0.721389	0.054211	6.504319	1.362191	0.049508

Example:

- Case 2: quadratic CEF – quadratic $\hat{f}$ show smaller bias, but higher variance than the linear $\hat{f}$ (bias exploded).

(b) Quadratic CEF

$$f(x) = x + x^2$$

```
sum2 = summary(training2)
sum2.loc[sum2['x']==x0,['mse1','var1','bias1','mse2','var2','bias2']] # if want latex: .
```

✓ 0.0s

	mse1	var1	bias1	mse2	var2	bias2
7	5586.446295	0.568593	5569.770371	8.324109	1.367391	0.003468

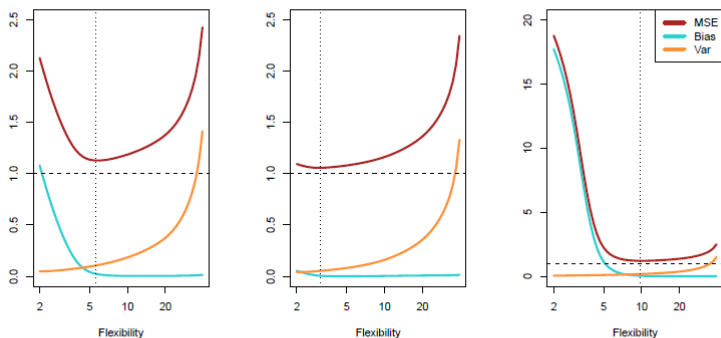


FIGURE 2.12. Squared bias (blue curve), variance (orange curve), $\text{Var}(\epsilon)$ (dashed line), and test MSE (red curve) for the three data sets in Figures 2.9–2.11. The vertical dotted line indicates the flexibility level corresponding to the smallest test MSE.

Idea 2: Training-Test Split

Standard econometric approach: emphasis on **parameters**

- fit the model
- evaluate model fit using R^2 , MSE
- look at goodness of fit for certain configurations of x 's
- consider augmenting/simplifying model

Statistical learning approach: data-driven, emphasis on **prediction**

- fit alt. models (different orders of p) on “training sample”
- evaluate models on “test sample”
 - select on ***out of sample performance*** (test MSE)
- model selection is data driven; can be automated

Example 1: Classification Problems

- $y \in \{y^1, y^2, \dots, y^G\}$ e.g. occupation
- model designed to assign some value $\hat{y}$ given predictors x
 - e.g.: have to “target” an advertising campaign, or assign an occupation code to a person
- For each obs i we can check if $y_i = \hat{y}_i$ (correct classification)
- Error rate = $\frac{1}{N} \sum_i 1(y_i \neq \hat{y}_i)$: analogue to MSE. For classification problem, you may also measure the quality of fit by F1-score

$$F_1 = \frac{2}{\text{recall}^{-1} + \text{precision}^{-1}}$$

- Benchmark classification (assign class with highest likelihood):

$$\hat{y}_i = y^g \text{ if } P(y_i = y^g | x_i) = \max_h P(y_i = y^h | x_i)$$

the “Bayes classifier”. If we knew $P(y_i = y^g | x_i)$ for each outcome g we would assign the highest probability outcome. This is the analogue of $E[y|x]$ for the 1-dimensional outcome case.

- Again: we don't know $P(y_i = y^g | x_i)$ and have to estimate it given the data!
- The statistical learning approach is to choose a classifier that has lowest error rate *in the **test sample** (or averaged test sample)*.

Example 2: k-Nearest Neighbors

Flexible modeling - KNN

- how could we use an existing data set (training data) to flexibly estimate $f(x)$? Example: KNN – “k nearest neighbors”
- Suppose x is one-dimensional. For given k and some x define $N_k(x) = \{x^1, x^2, \dots, x^k\}$ that are “closest” to x . Form estimate:

$$\hat{f}^k(x) = \frac{1}{K} \sum_{i \in N_k(x)} y_i$$

- Now we can compare performance of alternative choices of k on the **test sample**.
- And how would you select k - the number of neighbors? Also, data driven!

Re-sampling methods (ISLR, Chapter 5)

- in most cases we don't have a pre-identified test data set
- 3 basic approaches

1 *HOLDOUT SAMPLE*

- Designate a subsample of the data as the “training” data set, and the remainder as the test data set. The test data set is called a holdout (or holdback) sample, since you are NOT using it to estimate the model, **only to evaluate** alternative models.
- Two problems:
 - a) different holdout samples may yield different answers
 - b) the estimated model may be imprecise – using only 1/2 sample, we expect the standard errors to be $\sim \sqrt{2}$ times bigger than with the whole sample

2 LOOCV - Leave-one-out cross validation.

- For a sample of size N , hold out i^{th} observation.
- Estimate the model, predict $\hat{y}_i$, form $(y_i - \hat{y}_i)^2$
- Repeat for $i = 1 \dots N$ and form $\frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$
- Advantages:
 - a) average across all the LOO sample results $\Rightarrow$ only 1 “answer”
 - b) we use $N - 1$ observations each time
- BUT - there is a subtle disadvantage:
 - the LOO samples are very highly correlated with each other – get nearly the same answer as the within-sample MSE
 - LOOCV can result in **over-fitting**

this leads to the 3rd “intermediate” choice

3 Cross Validation

k-FOLD CROSS VALIDATION

- choose $k = 5$ or 10
- divide the data into k *random subsets*.
- for $j = 1 \dots k$: select the j – *th* subset as the holdout sample, fit the model, construct MSE_j on the hold out sample.
- construct $CV_k = \frac{1}{k} \sum_j MSE_j$

Advantages of *k*-FOLD CV:

- 1) only fit the model k times
- 2) different random partitions will yield slightly different answers, but with $k = 5, 10$ the differences tend to be very small
- 3) subsamples are correlated BUT less so than in the LOOCV case
⇒ less over-fitting

k-fold CV can be used in lots of settings, e.g. classification settings....

Idea 3: Regularization

A very general idea: “regularization”

- very large set of potential x 's
- what if we try to minimize: $SSR + \lambda \sum_j \hat{\beta}_j^2$ (penalty function)
- for given λ : find $\hat{\beta}(\lambda)$
 - now evaluate out-of-sample performance for each λ
- find best λ
- More on this in the next lecture.